

DESAFÍO TÉCNICO · DATA SCIENCE

Prevención de fraude orientada a ganancia

Un modelo que estima el riesgo de cada transacción y una política que decide con el costo de cada error.

Modelo, informe y respuestas a los cinco puntos del desafío



Juanfe Martínez

DATA SCIENCE · MACHINE LEARNING · PREVENCIÓN DE FRAUDE

Dataset · 150.000 transacciones

Código ·

github.com/JuanFeDS/fraud_shipping

Cómo leer este documento

Cada sección responde un punto del enunciado. Los títulos son enlaces.

PUNTO DEL DESAFÍO	SECCIÓN	PÁGINA
1· Modelo	Modelo y código: ficha técnica, enlaces y cómo ejecutarlo	3
2· Informe	Hipótesis	4
	Análisis y transformaciones del dataset	5
	Modelos utilizados	6
	Evaluación	7
	Conclusión	8
3· Laboratorio y producción	Que el laboratorio se parezca a producción	9
4· Causas	Si producción rinde distinto: las causas más probables	10
5· Puesta en producción	Poner el modelo en producción	11

El modelo entrenado y cómo reproducirlo

El código completo (notebooks, pipeline productivo y tests) está en github.com/JuanFeDS/fraud_shipping. Los resultados se reproducen con los pasos de esta página.

Ficha técnica

Modelo	LightGBM tuneado con 19 variables: las originales, tasa y frecuencia de <code>j</code> , país agrupado y hora
Política	Rechazar si $P(\text{fraude}) \geq 0,20$, el umbral que sale de la matriz de costos (+25% / -100%)
Resultado	78,9% de la ganancia ideal (IC95 de 77,6% a 80,1%) y +232 mil frente a aprobar todo
Validación	CV de 5 folds out-of-fold y validación semanal con solo el pasado (~77% por semana)
Código	Python 3.12 y Poetry; un único pipeline valida, puntúa y sirve; 91 tests (100% de cobertura) en CI
Producción	Model registry en MLflow; API en Cloud Run (28 ms p99) que registra cada decisión
Servicios desplegados	MLflow: mlflow-1027826425795.us-east1.run.app (solo lectura, credenciales a solicitud) API: api-fraude-1027826425795.us-east1.run.app/docs (/predecir con API key a solicitud)

Cómo ejecutarlo

```
poetry install · instala el entorno desde el lock
notebooks/01 → 07 · el análisis completo, en orden (el 04 tarda ~1,5 h por la búsqueda de hiperparámetros)
poetry run python scripts/validate_pipeline.py · reproduce el 78,9% con CV de 5 folds (~2 min)
poetry run python scripts/train.py · entrena el pipeline con todo el dataset
poetry run python scripts/predict.py --input <csv> --output <csv> · scoring batch
poetry run pytest · tests y cobertura
```

Cada decisión tiene un costo: aprobar un fraude o rechazar una venta

Introducción

El objetivo es construir un **modelo que prediga si una transacción es fraudulenta** y usarlo para decidir de forma que se **maximice la ganancia** de la empresa. Para entrenarlo y evaluarlo se cuenta con **150.000 transacciones etiquetadas** (8 de marzo al 21 de abril de 2020, 5% de fraude). Resultado de cada decisión:

DECISIÓN	TRANSACCIÓN LEGÍTIMA	TRANSACCIÓN FRAUDULENTO
Aprobar	+25% del monto	-100% del monto
Rechazar	0 · se pierde la venta	0 · se evita la pérdida

Aprobar un fraude cuesta cuatro veces lo que deja una venta legítima del mismo monto. Por eso se evalúa con **ganancia**, como porcentaje de la **ganancia ideal** (aprobar todas las legítimas y ningún fraude), y no con accuracy: aprobar todo acierta el 95% y deja pasar 547 mil en fraude. La solución tiene dos piezas: un **modelo** que estima $P(\text{fraude})$ y una **política** que rechaza cuando esa probabilidad supera un umbral elegido según el costo. Montos en la unidad monetaria del dataset.

Hipótesis y desarrollo

Seis hipótesis de negocio sobre el fraude en pagos online guiaron el análisis y la construcción del modelo:

#	HIPÓTESIS	RESULTADO
H1	Acertar más no es ganar más: el umbral de rechazo debería estar muy por debajo del 50%	PREMISA Se deduce de los costos: corte teórico 20%. Con 50%, 4,7 puntos menos
H2	El fraude busca montos altos	CONFIRMADA El 20% más caro tiene 7,3% de fraude (4–5% el resto) y el 82% de la pérdida, con el 67% del volumen
H3	Menos historial, más fraude	CON MATICES En f l m el cuartil más bajo triplica el fraude; en d e h , menos. Que sean historial es una interpretación
H4	El fraude tiene horarios	CONFIRMADA 0–4 h: 10% a 17% de fraude, con el 3,2% del volumen; quitar hora cuesta 0,26 puntos
H5	El score de riesgo ayuda, pero no alcanza solo	CONFIRMADA Como regla única deja el 67,4%; el modelo que lo combina con el resto llega al 78,9%
H6	El fraude se repite por categoría (j , 8.324 categorías)	CON MATICES Hay señal, pero en su forma original el modelo la memorizaba

Fraude según **o** (la segunda variable del modelo en valor; "vacío" = sin dato)

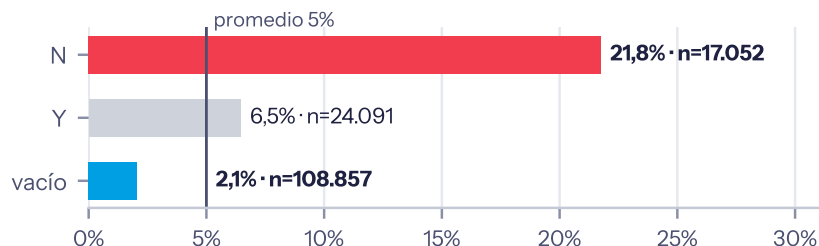


Figura 1 · El dato vacío en **o** es la señal de menor riesgo (2,1%) y **N** multiplica por 4 la tasa global: desordenar **o** cuesta 6,3 puntos de ganancia.

De datos crudos a señales que no memorizan

150k

transacciones, 0 duplicados

5%

de fraude (7.500)

45 días

marzo-abril 2020

19columnas: 15 anonimizadas,
fecha, monto, score y
etiqueta

Hallazgos del análisis

5% exacto de fraude	→ probable muestreo: con la tasa real hay que corregir las probabilidades
72,6% de datos vacíos en o	→ el vacío es información (en o, la señal de menor riesgo): no se imputa
País (g): 74% Brasil · 21% Argentina	→ países con 1 o 2 transacciones no permiten estimar su riesgo
Fraude diario entre 2,2% y 6,6%	→ el riesgo varía en el tiempo y hay que monitorearlo
Score: sin señales de filtración	→ relación gradual e imperfecta con el fraude; el resto de las variables explica solo el 16% de su valor

Transformaciones

Se construyeron 18 variables. Cada una se evaluó por su señal individual (con una variable aleatoria como piso de ruido) y por su aporte al modelo, comparando conjuntos fold a fold. El conjunto final agrega cuatro (el perfil o/n/p se descartó) y reemplaza a j y g crudas:

VARIABLE	CONSTRUCCIÓN	POR QUÉ
Tasa de fraude de j	Target encoding suavizado hacia la tasa global, calculado sin la etiqueta de la propia fila	Conserva la señal de j sin memorizar (H6)
Frecuencia de j	Transacciones de la categoría en entrenamiento	2,8% de fraude en categorías de 1 caso, 7,0% en las de más de mil
Perfil o/n/p	Combinación de las tres, con el vacío como categoría	AUC univariado 0,799, pero o ya lo captura: descartado
País agrupado	Países con menos de 100 transacciones → "Otros"	Evita tasas de países con 1 o 2 casos
Hora	Hora del día	Riesgo de la madrugada (H4); quitarla cuesta 0,26

Las otras 13 (ratios de historial, montos redondos, actividad reciente, día de la semana) no mejoraron el modelo: daban el mismo resultado con 33 variables en lugar de 19. La mejora grande es la tasa de j; hora y país aportan poco por separado, porque LightGBM ya combinaba las originales.

SIN LA PROPIA ETIQUETA (OUT-OF-FOLD)

En el pipeline final, las variables que resumen otras transacciones (tasa y frecuencia de j, países frecuentes) se aprenden solo con los folds de entrenamiento. Dentro de él, la tasa de j usa folds internos para que ninguna fila use su propia etiqueta: calculada de forma ingenua parecía mucho más fuerte (AUC univariado 0,783 frente a 0,645), y esa diferencia era memorización. Con folds aleatorios no es un cálculo temporal: una tasa puede usar etiquetas posteriores. Los vacíos no se imputan: LightGBM los trata de forma nativa.

Un cambio por vez, **contra una referencia**

Validación cruzada estratificada de 5 folds con predicciones out-of-fold; en cada configuración, el umbral se elige sobre la curva de ganancia. Cada etapa cambió un solo elemento y se comparó contra la anterior en los mismos folds, fold a fold.

ETAPA	RESULTADO (% DE LA GANANCIA IDEAL)
Referencias sin ML	Aprobar todo 63,4% · rechazar desde score 90 67,4%
Baseline	LightGBM por defecto, variables originales: 77,7%
Variables	Quitar j cruda y agregar las transformaciones, todavía con el perfil o/n/p: 79,0% (+1,2 puntos, mejor en los 5 folds)
Algoritmos	Cinco algoritmos con esas variables y parámetros por defecto (figura 2)
Ajuste de hiperparámetros	Optuna, 50 combinaciones: +0,4 puntos con otra partición, IC95 de -0,5 a +1,3 (la búsqueda prometía +0,7)
Pesos por monto	No mejoran: con peso proporcional al monto la ganancia baja 0,5 puntos
Simplificación	Quitar el perfil o/n/p no cambia la ganancia (+0,1, mejor en solo 2 de 5 folds): una variable menos

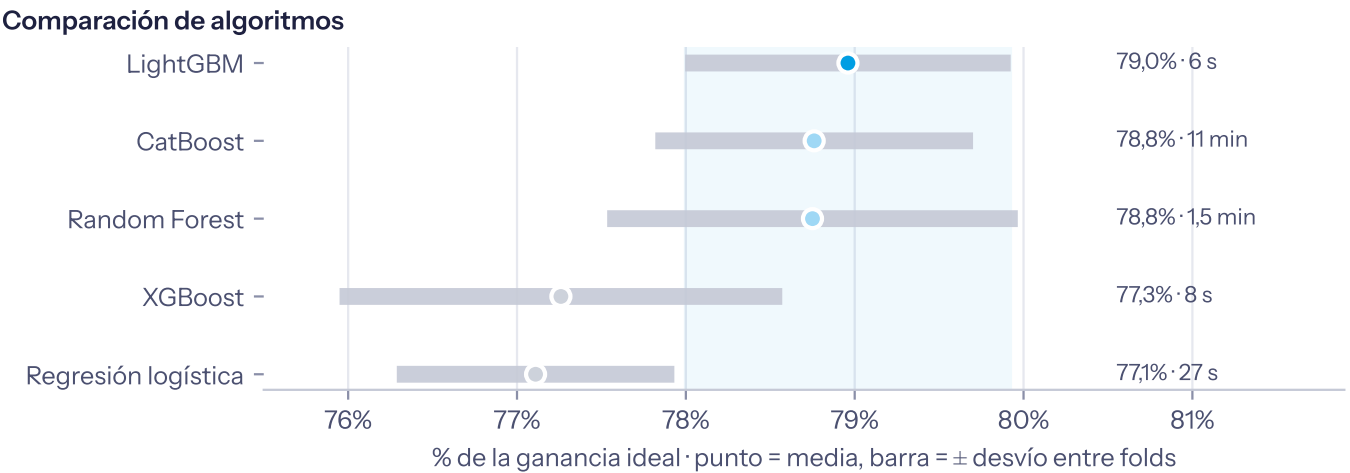


Figura 2 · LightGBM, CatBoost y Random Forest empatan dentro de la variación entre folds.

Se eligió **LightGBM** por su desempeño equivalente, una validación 100 veces más rápida que CatBoost y manejo nativo de nulos y categóricas.

Solución final: modelo + política

Modelo	LightGBM ajustado (481 árboles, Learning_rate 0,019, num_Leaves 172, hojas de al menos 262 casos) que estima P(fraude) con 19 variables
Política	Rechazar si $P(\text{fraude}) \geq 0,20$, el umbral teórico: sale de los costos, sin estimar nada, y los umbrales alternativos probados no se distinguen de él
Entrega	Pipeline único (variables + modelo + umbral) versionado en MLflow y servido por una API en Cloud Run

Medido en dinero, fuera de muestra

Todas las métricas se calculan con predicciones **out-of-fold**: cada transacción la evalúa un modelo que no la usó para aprender. El pipeline final se validó con otra partición de cinco folds: **78,9%** de la ganancia ideal, IC95 de 77,6% a 80,1% (ROC-AUC 0,889, PR-AUC 0,474). Prediciendo cada semana con el pasado varía más: **~77%, entre 74% y 80%**.

Ganancia de cada alternativa



Figura 3 • +15,5 puntos sobre aprobar todo y +11,5 sobre la regla por score.

Decidir con el costo pesa más que el algoritmo

POLÍTICA ELEGIDA • UMBRAL 20%

78,9%

de la ganancia ideal

CORTE POR DEFECTO • 50%

74,2%

el de un clasificador: aprueba demasiados fraudes

DIFERENCIA

-4,7 puntos

con el mismo modelo

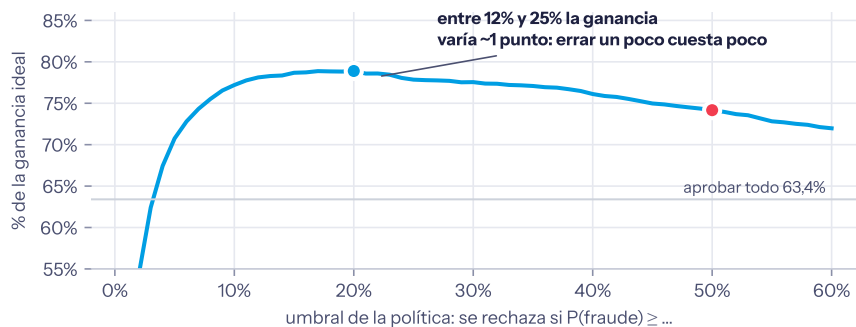


Figura 4 • El corte teórico, 20% (el monto se cancela), es también el óptimo en la CV: 78,9%.

Resultado en dinero

+232 mil

frente a aprobar todo, sobre 150.000 transacciones de 45 días (1.180 mil frente a 948 mil)

59%

del dinero en fraude evitado (325 de 547 mil), detectando el 48,5% de los fraudes

94,5%

de las transacciones aprobadas; 3% de las legítimas rechazadas

Robustez y supuestos

- **Calibración:** 31% predicho y observado en el decil más alto; cerca del umbral, 17% frente a 19% por monto. En el tiempo subestima; recalibrado con isotonic, +0,9 puntos.
- **Umbral:** frente a 15%, +0,2 (IC95 -0,3 a +0,7); elegido con semanas previas (0,12), no se distingue de 0,20.
- **Tiempo:** con solo el pasado, 75,3%, 73,7%, 77,1% y 79,5% en las 4 últimas semanas (-1,2 frente a la CV en las mismas filas; -0,4 sin la primera, entrenada con 60.000 transacciones); con ajuste y umbral del pasado, 80,0% en la última.

Detectar el fraude **que más cuesta**

La solución alcanza el **78,9% de la ganancia ideal**, frente al 63,4% de aprobar todo y al 67,4% de la regla basada en el score: **232 mil más** sobre las 150.000 transacciones evaluadas fuera de muestra, evitando el 59% del dinero perdido por fraude y aprobando el 94,5% de los pagos.

Qué explica el resultado

1 Medir en ganancia y decidir con el costo

Derivar el corte del costo vale 4,7 puntos frente al 50% por defecto de un clasificador; elegir entre los mejores algoritmos, 0,2.

2 Representar bien la historia de las categorías

La variable **j** cruda empeoraba el modelo; como tasa de fraude out-of-fold fue la mayor mejora.

3 Acotar el optimismo de la validación

El tuning prometía +0,7 y aporta +0,4, sin distinguirse de cero. Prediciendo semanas futuras, el modelo pierde 1,2 puntos frente a la CV en las mismas filas (0,4 con historia suficiente).

Limitaciones

- **Variables anonimizadas:** las lecturas de negocio (historial, perfiles, país) son interpretaciones. El monto parece estar en una moneda común: su escala es similar entre países (mediana 18 en Brasil, 29 en Argentina).
- **Datos muestreados y atípicos:** 5% exacto de fraude y 45 días del inicio de la pandemia; con la tasa real, las probabilidades deben corregirse por la tasa base.
- **Dependencia del score:** es la variable más valiosa (desordenarla cuesta 8,3 puntos) y la raíz de los errores más caros. Se asume disponible antes de decidir; sin ella, el modelo da 77,0% (-1,9 puntos).
- **Sesgo de selección y fricción:** si el dataset excluye lo que el sistema actual ya rechaza, el modelo no vio ese tráfico. Y la matriz de costos no mide la fricción de rechazar 4.612 compras legítimas (abandono, reclamos).

Próximos pasos

- Confirmar que el score está disponible al momento de decidir y ponerlo en producción en modo sombra.
- Recalibrar las probabilidades con etiquetas maduras recientes (isotonic: +0,9 puntos en el tiempo), con el umbral fijo en 0,20, y monitorear el drift.
- Pedir 3DS en la franja 0,15–0,40 (5% del tráfico): de +2,6 a +7,0 puntos según el abandono y el fraude que frene.
- Estudiar los fraudes con score medio, el principal punto ciego del modelo.

El valor no vino de un algoritmo en particular, sino de medir en dinero, decidir con el costo de cada error y medir el resultado sin la propia etiqueta de cada transacción.

CÓDIGO, NOTEBOOKS Y REPORTES DE CADA ETAPA • [GITHUB.COM/JUANFEDS/FRAUD_SHIPPING](https://github.com/JUANFEDS/FRAUD_SHIPPING)

Que el laboratorio se parezca a producción

La evaluación debe reproducir las condiciones en las que el modelo va a decidir: los mismos datos disponibles, el mismo código y la misma métrica de negocio.

1 · Validar como se va a usar el modelo

1 Variables con estado aprendidas solo en entrenamiento

Tasa y frecuencia de `j` y países frecuentes se aprenden en entrenamiento y se aplican tal cual a los nuevos; la tasa de `j`, out-of-fold. Así validado, el pipeline reproduce el laboratorio: **78,9% ± 1,3** de la ganancia ideal.

2 Evaluar en el tiempo, no solo con folds aleatorios

Con folds aleatorios, una tasa de `j` puede usar etiquetas posteriores. Entrenando solo con el pasado, el modelo final da 75,3%, 73,7%, 77,1% y 79,5% en las cuatro últimas semanas (-1,2 frente a la CV aleatoria en las mismas filas; -0,4 sin la primera, entrenada con solo 60.000 transacciones); con ajuste y umbral elegidos solo con el pasado, 80,0% en la última.

3 No evaluar con los datos que se usaron para elegir

El tuning parecía aportar +0,7 puntos con los folds de la búsqueda y +0,4 con otra partición, que luego decidió también pesos, variables y umbral: reusada, deja de ser nueva. Hay que elegir con datos anteriores a los de evaluación.

4 Etiquetas maduras

Un contracargo puede llegar semanas después: con transacciones recientes, muchos fraudes todavía figuran como legítimos y la evaluación sale optimista. Hay que fijar una **ventana de maduración** (por ejemplo, 60 a 90 días) y usar solo transacciones que la cumplieron.

5 Misma frescura de las variables

En producción, la tasa de `j` solo puede usar etiquetas maduras: la validación debe simular ese desfase y la tabla refrescarse con una cadencia definida.

2 · Medir lo que importa al negocio

- **Ganancia, no accuracy:** aprobar todo acierta el 95% y obtiene solo el 63,4% de la ganancia ideal.
- **Umbral derivado del costo:** aprobar conviene si $p < 20\%$, el óptimo con CV aleatoria. Elegido solo con semanas previas, otro umbral (0,12) no se distingue de 20%. Si cambia la tasa de fraude, se corrigen las probabilidades, no el umbral (π : tasa de entrenamiento; π' : vigente): $p' = p_a / (p_a + (1-p)b)$, con $a = \pi'/\pi$ y $b = (1-\pi')/(1-\pi)$; o se recalibran: con isotonic sobre semanas previas, +0,9 puntos en el tiempo.
- **Con su incertidumbre:** 78,9% (IC95 de 77,6% a 80,1%), pero ~2,5 puntos de variación entre semanas: las alertas deben usar esa variación para no confundir ruido con degradación.

3 · El mismo código y el mismo entorno

- Un único pipeline valida, puntúa en batch y responde en la API, con tests de que una transacción recibe la misma probabilidad sola, en lote o por la API.
- Versiones de librerías desde un único lock: la imagen de la API no traía `pyarrow`, necesario para cargar el modelo, porque sus dependencias se mantenían a mano.

4 · Verificar los supuestos sobre los datos

- **Disponibilidad al decidir:** confirmar que todas las variables, en especial `score`, existen antes de aprobar. Es la más valiosa (desordenarla cuesta 8,3 puntos); sin ella, el modelo da 77,0%.
- **Sesgo de selección:** si el dataset excluye lo que el sistema actual ya rechazó, el modelo aprendió sobre una población filtrada y, al reemplazar ese filtro, verá casos nuevos. Hay que confirmarlo y medir en sombra sobre todo el tráfico.
- **Representatividad:** 5% exacto de fraude y 45 días de pandemia; comparar con la tasa real.

Si producción rinde distinto: las causas más probables

Ordenadas de más a menos probable para una diferencia **grande**, con la señal que permite distinguir cada una.

#	CAUSA	POR QUÉ PASA	CÓMO SE DETECTA
1	Variables no disponibles a tiempo (leakage)	Si <code>score</code> u otra variable incorpora información posterior a la compra, el modelo aprendió de algo que en producción no existe. El análisis no encontró señales, pero no se puede descartar sin saber cómo se calcula.	Caída brusca desde el primer día; la distribución de la variable en producción difiere de la de entrenamiento.
2	Población distinta (sesgo de selección)	Si el dataset excluye lo que el sistema actual rechaza, el modelo nunca vio ese tráfico y sus probabilidades ahí no están calibradas.	Distribución de probabilidades y tasa de rechazo muy distintas a las del laboratorio.
3	Diferencias entre entrenar y servir	Tipos, nulos, categorías nuevas o agregados con otra ventana. En este proyecto aparecieron dos casos, ya resueltos: numéricas nulas que llegaban como texto con una sola fila, y categorías no vistas que se volvían nulo sin aviso. Y la zona horaria: la API exige la fecha con zona y la convierte a la del dataset (UTC-3, supuesta), para que la hora del día signifique lo mismo que al entrenar.	La misma transacción recibe distinta probabilidad en batch y en la API.
4	Otra tasa de fraude o de montos	El 5% del dataset probablemente no es el real, y la tasa diaria osciló entre 2,2% y 6,6%. Las probabilidades quedan desplazadas: la ganancia cae aunque el modelo ordene bien.	El AUC se mantiene y la ganancia cae; corregir las probabilidades por la tasa vigente la recupera.
5	El fraude se adapta al modelo	Los defraudadores prueban y aprenden qué pasa el control; además, el modelo cambia la población que ve (lo que rechaza no genera etiquetas). El entrenamiento corresponde a un período atípico.	Degradación gradual y concentrada en ciertos segmentos (categorías de <code>j</code> , países, montos).
6	Cambios en sistemas de los que depende	Si el sistema que genera <code>score</code> se recalibra o se reemplaza, la variable más importante cambia de significado sin que cambie el código del modelo.	Salto en la distribución de <code>score</code> sin cambios en el modelo.
7	Fallas operativas	Timeouts o caídas resueltos con una política de respaldo (por ejemplo, aprobar todo) se ven como "peor performance" aunque el modelo no haya decidido.	Proporción de decisiones tomadas por el respaldo; latencia p99.
8	Medición incompleta	Etiquetas que todavía no maduraron, transacciones rechazadas sin etiqueta y ventanas cortas (la ganancia varía ~2,5 puntos entre semanas).	La ganancia medida mejora a medida que maduran las etiquetas del mismo período.

Poner el modelo en producción

Los tres primeros pasos están implementados en el proyecto; los demás son los que faltan para operar con tráfico real.

- 1 Empaquetar como pipeline reproducible** HECHO
 Variables, modelo y umbral en un único artefacto (`FraudPipeline`), con 91 tests unitarios y de integración que corren en CI en cada push.
- 2 Validar, registrar y versionar** HECHO
 Todo queda en MLflow y el rollback es mover el alias `champion`. Un **gate de promoción** exige una validación del mismo modelo y datos y no más de 0,5 puntos bajo el champion; falta comparar a ambos sobre el mismo holdout reciente y exigir el mismo commit.
- 3 Servir** HECHO
 API en Cloud Run con validación del input (fecha con zona horaria) y API key que falla cerrada; la imagen incluye el modelo y `/salud` informa la versión. Cada decisión queda en Cloud Logging con su id, probabilidad, versión y latencia. También scoring batch.
- 4 Definir latencia y respaldo** SIGUIENTE
 Medido en Cloud Run (1 vCPU, 300 requests): el servidor decide en **20 ms p50 y 28 ms p99**; fuera de la región, la red suma ~80 ms. Falta fijar el SLA del checkout, probar con carga, mantener una instancia caliente (arranque en frío: 3,8 s) y definir el respaldo (recuadro).
- 5 Desplegar gradualmente** SIGUIENTE
 Modo sombra primero: el modelo puntúa el tráfico real sin decidir. Luego champion/challenger con criterios de éxito definidos de antemano (ganancia, tasa de rechazo, fraudes detectados) y comparación pareada (recuadro).
- 6 Una tercera opción entre aprobar y rechazar** SIGUIENTE
 La política rechaza el 3% de las legítimas (93 mil de margen) sin medir la fricción. Simulado con 10% de abandono y 85% de fraude frenado, **pedir 3DS** al 5% de las transacciones, entre ~0,15 y ~0,40, gana +5,6 puntos en la última semana (IC95 de +4,1 a +7,4). Hay que medir esos supuestos en producción.
- 7 Monitorear** SIGUIENTE
 Sin esperar etiquetas: tasa de rechazo, distribución de probabilidades, drift de `score`, `monto`, `o` y país, y proporción de categorías de `j` no vistas. Con etiquetas maduras: fraude detectado y ganancia realizada frente a la esperada. Alertas por rango.
- 8 Reentrenar y automatizar** SIGUIENTE
 Con etiquetas maduras, cadencia fija o ante drift, recalibrando las probabilidades. Automatizar el despliegue al promover una versión (el CI ya existe) y SHAP por decisión para reclamos.

SI EL MODELO SE CAE: ¿APROBAR O RECHAZAR TODO?

Rechazar todo deja el 0% de la ganancia; aprobar todo, el 63,4%, pero sin ningún freno. El mejor respaldo es el sistema o la regla vigente (la regla por score deja el 67,4%). Un límite de monto no sirve: rechazar lo más caro deja entre 28% y 50%, porque ahí también está el margen. Si lo que falta es el score, un modelo sin él da 77,0%. La verificación de la API key, en cambio, debe fallar cerrada: si falta su configuración, la API no arranca.

CHAMPION/CHALLENGER: EL LÍMITE ES LA MADURACIÓN, NO EL VOLUMEN

Con grupos independientes, detectar 1 punto (95% de confianza, 80% de potencia) requiere ~800.000 transacciones por grupo: a miles por segundo, son minutos. El cuello de botella es la maduración de los contracargos (60 a 90 días) y cubrir varias semanas, porque la ganancia varía ~2,5 puntos entre ellas. Conviene puntuar las mismas transacciones con ambos modelos (en sombra) y comparar semana a semana, dejando una pequeña fracción aprobada al azar para medir sin el sesgo de las rechazadas.